Bank of a Vibe Code

A Hardened Banking Application — Built in 5 Days by a CS Junior with No Prior Python Experience

By Jaden Gomez

> *This project was built in active collaboration with AI as a development tool. All architectural decisions, security validations, threat modeling, and iterative fixes were directed, challenged, and verified manually.*

The Short Version

Built a full-stack banking web application with production-grade security from scratch in ~5 days (3–5 hours/day, around a full class schedule and rugby practice). Prior experience: Java and basic SQL. No Python, no Flask, no web dev, no security background.

The app passed an OWASP ZAP Dynamic Application Security Testing scan with **0 real high-severity vulnerabilities**, has a live CI/CD pipeline, runs on a real Azure VM behind Nginx with HTTPS, and was audited adversarially for logic flaws that automated scanners can't find.

Live domain: bankofvibecode.com

Tech Stack

Layer	Technology
Backend	Python / Flask
WSGI Server	Gunicorn (4 workers)
Database	MySQL
Reverse Proxy	Nginx (HTTPS, HSTS, CSP headers)
SSL	Let's Encrypt
Containerization	Docker
Cloud	Azure VM
CI/CD	GitHub Actions

Layer	Technology
Security Testing	OWASP ZAP (automated, runs on every push)

Why a Banking App

Intentional. A banking application forces you to think about the problems that actually matter in production systems:

- **Money handling** — floating point arithmetic breaks financial math. `float` is wrong. `Decimal` is right.
- **Concurrency** — two simultaneous withdrawal requests can both pass a balance check before either one writes. Race conditions aren't theoretical.
- **Abuse prevention** — rate limiting, account lockouts, and CSRF protection aren't optional features in a financial context.
- **Security from day one** — building the app *around* the OWASP Top 10 threat model rather than patching it afterward.

Most first projects are todo apps. This one was designed to fail in interesting ways so those failure modes could be fixed.

Security Architecture

Authentication & Session Management

- Passwords hashed with Werkzeug (salted bcrypt) — not even the developer can see plaintext passwords
- Per-account lockout after 5 failed attempts (15-minute lockout), independent of IP
- IP-based rate limiting on login (5/min), register (5/min), deposit (10/min), withdraw (10/min)
- Session fixation protection — `session.clear()` before setting session data on login
- Session hardening: `HTTPOnly`, `Secure`, `SameSite=Lax` cookie flags
- 30-minute absolute session timeout

Financial Logic

- **Atomic withdrawal transactions** — single SQL `UPDATE ... WHERE balance >= amount`

with `rowcount` check eliminates the Time-of-Check/Time-of-Use race condition. No read-then-update pattern.

- **`Decimal` throughout** — every money input converted from form data using `Decimal(request.form["amount"])` with `InvalidOperation` handling. Zero float arithmetic.
- Transaction caps: $10,000 max per deposit/withdrawal
- Overdraft protection at the database level, not the application level

Input & Request Validation

- Parameterized queries everywhere — SQL injection structurally impossible
- CSRF protection via Flask-WTF on every form (login, register, deposit, withdraw)
- Input length caps: full name (100), username (50), password (128)
- Account type validation on deposit/withdraw — only `"checking"` or `"savings"` accepted, anything else rejected before touching the database

HTTP Security Headers

```
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none';
frame-ancestors 'none'
```

Infrastructure Security

- Nginx as reverse proxy — Flask/Gunicorn never directly exposed
- HTTPS enforced via Let's Encrypt SSL cert
- HSTS configured at the Nginx layer
- Docker containerization — app isolated from host environment
- Secrets managed via `.env` / environment variables, never in source code
- `.gitignore` prevents `.env` from reaching GitHub

Logging & Monitoring

- Rotating file logs (RotatingFileHandler) — startup, logins, failed attempts, deposits, withdrawals
- Real IP logging via `ProxyFix` — meaningful for fraud detection behind a reverse proxy

- Account lockout events logged separately from general failed attempts

---

CI/CD Pipeline

Every `git push` to `main`:

1. GitHub Actions connects to the Azure VM via SSH
2. Builds a fresh Docker image
3. Stops and removes the old container
4. Starts the new container with environment variables injected
5. Restarts Nginx
6. **Runs an OWASP ZAP baseline scan against the live domain automatically**

Security is not a manual step. It runs on every deployment.

OWASP ZAP Scan Results

Ran OWASP ZAP 2.17.0 against the live application.

Result: 0 high-severity vulnerabilities. 0 confirmed exploits.

The one red flag ZAP raised was a **SQL injection false positive** (confidence: Medium). ZAP flagged it because the login endpoint returned different responses for different inputs — which is expected behavior when an account lockout system is working correctly. The underlying query is parameterized, which makes SQL injection structurally impossible regardless of input.

ZAP also confirmed:

- Rate limiting functional (429 responses observed)
- No authentication bypass
- No XSS vectors
- No CSRF gaps

The Audit — What Got Found and Fixed

After building the initial version, the app was subjected to a full adversarial logic audit focused on vulnerabilities automated scanners miss. Issues found and resolved across two rounds:

Issue	Status
`/test500` debug route live in production	✅ Removed
Withdrawal race condition (TOCTOU)	✅ Fixed with atomic SQL
`float` used for all money math	✅ Replaced with `Decimal` throughout
Rate limiting only on `/login`	✅ Extended to `/register`, `/deposit`, `/withdraw`
`SELECT *` passing password hash to templates	✅ Fixed — only required columns fetched
No input length limits	✅ Added server-side caps
No deposit amount cap	✅ $10,000 max enforced
IP-only rate limiting bypassable by botnet	✅ Added per-account lockout
Session fixation on login	✅ Fixed with `session.clear()`
Duplicate rate limiter decorator	✅ Cleaned up
ZAP pipeline `continue-on-error: true`	✅ Removed — pipeline now enforces scan

What's Left (Honest)

Every project has a backlog. Current known items:

- **Server-side sessions** — client-side signed cookies are a valid pattern but server-side invalidation on logout is stronger for a banking context
- **CSP nonce migration** — `unsafe-inline` in `style-src` still present; full nonce implementation requires auditing all templates for inline styles first
- **Docker bridge networking** — currently on `--network host`; should move to a bridge network with explicit port exposure
- **Non-root VM deployment user** — dedicated limited deploy user is better practice than the default VM user
- **Username sanitization in logs** — hashed identifiers would be safer if logs are exfiltrated

These are infrastructure hardening items. None are exploitable vulnerabilities in the current state.

What I Learned

Going from zero Python to a deployed, security-audited web application taught me that security isn't a feature you add — it's a set of constraints you design around from the start. The OWASP Top 10 isn't a checklist, it's a map of how real systems actually fail.

The most valuable skill this project built wasn't Flask or Docker. It was learning to ask "how would someone break this?" before asking "does this work?"

Skills Demonstrated

`Python` `Flask` `MySQL` `Docker` `Nginx` `GitHub Actions` `Azure` `OWASP ZAP` `CI/CD` `Threat Modeling` `Parameterized Queries` `CSRF Protection` `Rate Limiting` `Session Security` `Linux` `SSL/TLS` `Gunicorn` `Defensive Programming`